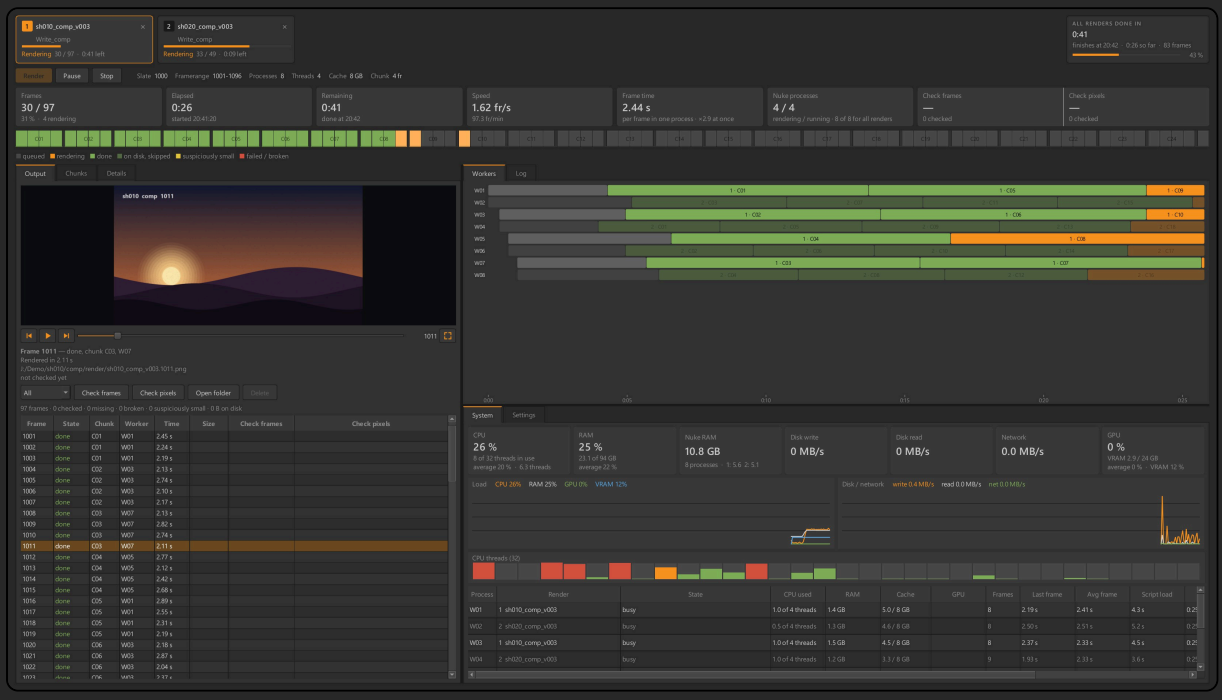
ANLEITUNG · VERSION 0.94.3 ALPHA

JKrender

Schnelles lokales Rendern für Nuke. Ein Render wird in Stücke (Chunks) geteilt, die mehrere Nuke-Prozesse gleichzeitig rechnen. Nach dem Render prüft JKrender jedes Frame auf der Platte.

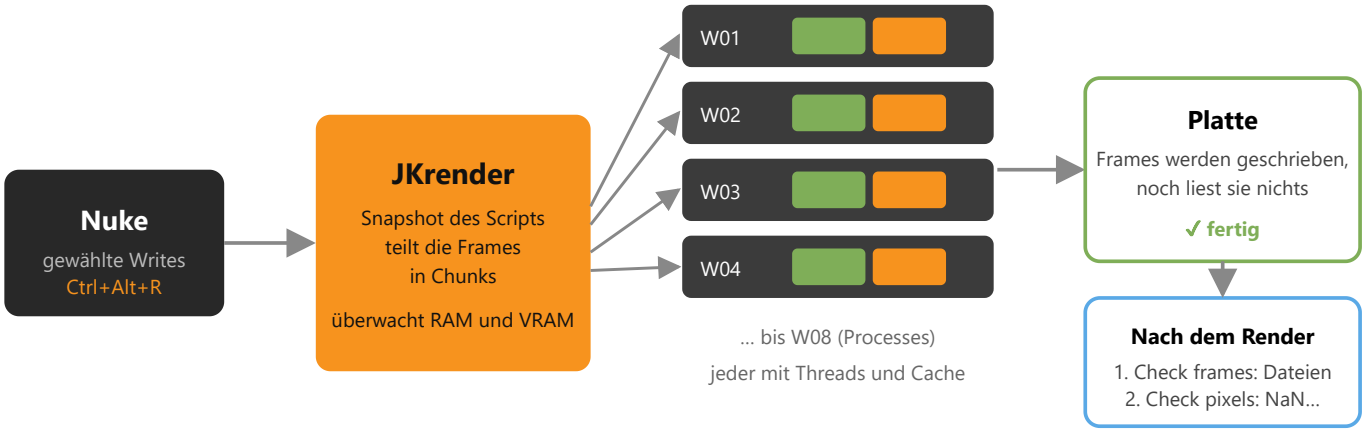


Inhalt

1 Wie JKrender arbeitet
2 Installation und Start aus Nuke
3 Übersicht des Fensters
4 Render-Karten und Steuerzeile
5 Kacheln und Frame-Karte
6 Panel Output: Vorschau und Frames
7 Panels Chunks und Details
8 Panels Workers und Log
9 Panel System
10 Settings: jede Einstellung
11 Einen Render einstellen
12 Test this script
13 RAM und VRAM im Blick
14 Prüfungen, Töne, Absturz, Daten
15 Maus und Tasten: Spickzettel
16 Wenn etwas nicht geht

1 Wie JKrender arbeitet

Ein normaler Nuke-Render läuft in einem Prozess, der das ganze Script Frame für Frame rechnet. Ein Nuke nutzt aber selten alle 32 Threads der CPU. JKrender startet deshalb **mehrere Nuke-Prozesse nebeneinander**. Jeder **öffnet das Script nur einmal** und bekommt dann Arbeitsstücke, sogenannte **Chunks**: zum Beispiel 10 Frames hintereinander.



Zum Ende hin werden die Chunks kleiner, so werden alle Prozesse etwa gleichzeitig fertig.

Drei Zahlen, die über die Geschwindigkeit entscheiden

Processes

Wie viele Nuke-Prozesse gleichzeitig laufen (für alle Renders zusammen). Mehr Prozesse schaffen mehr auf einmal, brauchen aber auch mehr RAM und VRAM.

Threads

Wie viele CPU-Threads jeder Prozess bekommt (Nuke `-m`). **Processes × Threads** sollte etwa die Thread-Zahl des PCs ergeben (hier 32).

Cache

Das Cache-Limit eines Prozesses in GB (Nuke `-c`). Ein Prozess nimmt in Wirklichkeit mehr. Wie viel, zeigt System › RAM.

> Standard sind **8 Prozesse × 4 Threads**. Schwere Comp-Scripts, die ein Nuke schon allein auslastet (Defocus, Denoise), sind auf **4 × 8** oft schneller und brauchen halb so viel RAM. Scripts, deren VRAM wächst, wollen weniger Prozesse. Was für dein Script am besten ist, misst **Test this script** (Kapitel 12).

Wo was liegt

Was	Wo
Einstellungen	`~/.nuke/JKrender/settings/settings.json` (jeder Benutzer hat seine eigenen). Ältere Versionen hatten sie in `~/.jkrender`; JKrender verschiebt sie beim ersten Start selbst.
Render-Logs	`~/.nuke/JKrender/log/`: ein Ordner pro Render (Snapshot des Scripts, Job, Log jedes Nuke-Prozesses und `report.json`). Ältere als 15 Tage werden gelöscht.
Testergebnisse	`~/.nuke/JKrender/settings/test_results`
Gerenderte Frames	Dort, wohin der Write sie schickt. JKrender löscht sie nie von selbst, nur mit der Taste Delete.

2 Installation und Start aus Nuke

Installation (einmal)

1. Kopiere den ganzen Ordner `JKrender` nach `%USERPROFILE%\.nuke\`. Als Zip von GitHub heißt er `JKrender-main`: benenne ihn in `JKrender` um.

2. Füge in `~/.nuke/init.py` diese Zeile hinzu (lege die Datei an, falls es sie nicht gibt):

```
nuke.pluginAddPath("./JKrender")
```

3. Starte Nuke neu. Beim ersten Start fragt JKrender, ob es numpy und OpenEXR für die EXR-Vorschau laden soll (etwa 15 MB). Sagst du nein, rendert es trotzdem, nur ohne EXR-Vorschau. Um wieder gefragt zu werden, lösche die Datei `lib/DECLINED`.

Optional: für Details und den Player von MOV/MP4 ffprobe/ffmpeg in den PATH legen (`winget install ffmpeg`). Nötig ist Nuke 16 (PySide6).

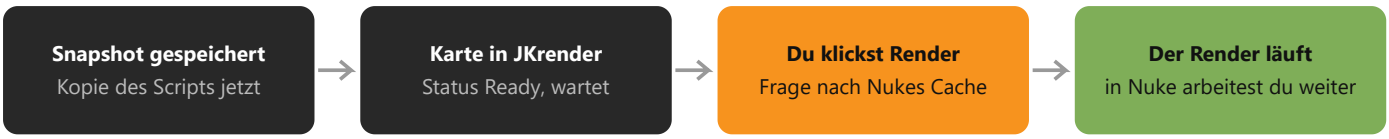
Neue Version: kopiere die neuen Dateien über die alten. Die Ordner `lib`, `log` und `settings` bleiben.

Das Menü JKrender in Nuke

Eintrag	Was er tut
Render Selected Write(s) `Ctrl+Alt+R`	Schickt die gewählten Writes (oder Groups mit Writes) an JKrender. Jeder Write bekommt eine eigene Karte. Ist das Fenster noch nicht offen, öffnet es sich.
Open Renders Folder	Öffnet den Ordner mit den Render-Logs.

Ist das Script nicht gespeichert, fragt es *Save it and render?*: gerendert wird die gespeicherte Fassung.

Was nach dem Senden passiert

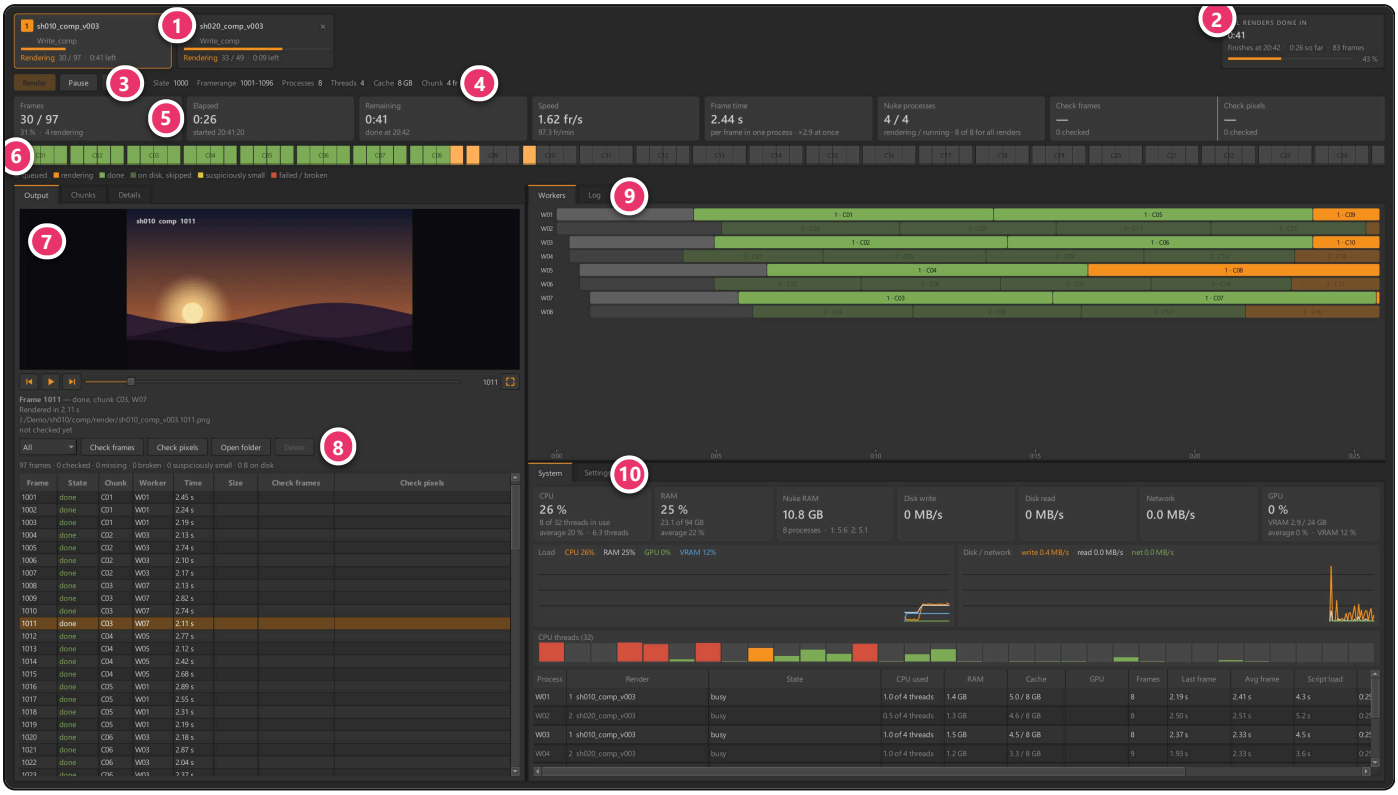


> **Snapshot:** gerendert wird das Script so, wie es beim Senden war. In Nuke kannst du sofort weiterarbeiten und speichern, der laufende Render merkt davon nichts. Willst du ein geändertes Script rendern, schicke es erneut aus Nuke: das neuere Senden ersetzt die alte Karte.

> **Frage nach dem Cache:** ist ein Nuke offen, fragt Render, ob dessen Buffer, Playback-Cache und Blink-Cache geleert werden sollen, damit der Render mehr Speicher hat. Die Standardantwort ist **No**, der Render startet in beiden Fällen. Der Disk-Cache bleibt. Nach einem Update von JKrender Nuke einmal neu starten, damit das geht.

3 Übersicht des Fensters

Oben sind die Render-Karten und die Steuerzeile, darunter die Zahlen und die Frame-Karte. Der Rest sind drei Panels: links **Output / Chunks / Details**, rechts oben **Workers / Log**, rechts unten **System / Settings**. Die Grenzen zwischen den Panels lassen sich ziehen.



1. Render-Karten: ein Write (Render) pro Karte. Ein Klick wählt ihn.
2. Wann alle Renders fertig sein sollten.
3. Render, Pause, Stop für den gewählten Render.
4. Slate, Framerange, Processes, Threads, Cache und Chunk dieses Renders (nur zum Lesen).
5. Kacheln mit Zahlen: Frames, Zeit, Tempo, Prozesse, Prüfungen.
6. Die Frame-Karte nach Chunks. Ein Klick auf ein Frame zeigt es.
7. Output: Vorschau eines Frames mit Abspielen.
8. Output: Filter, Prüfungen, Ordner, Löschen und die Frame-Liste.
9. Workers: Zeitleiste jedes Nuke-Prozesses.
10. System: Auslastung des PCs und Tabelle der Prozesse.

4 Render-Karten und Steuerzeile

Eine Render-Karte

Jeder gesendete Write ist eine Karte: Nummer, Script-Name, Write-Name, Fortschrittsbalken und Status (Ready, Waiting, Rendering, Paused, Checking output, Done, Failed, Stopped). Ein **Klick** wählt die Karte, das ganze Fenster zeigt dann diesen Render. **Shift+Klick** wählt mehrere Karten; Render, Pause und Stop gelten dann für alle, Render again auch für fertige (es fragt einmal für alle). Das × schließt den Render, die Frames auf der Platte bleiben.

Renders teilen sich ein Budget an Prozessen. Nebeneinander laufen höchstens so viele, wie Settings *Max renders at once* sagt (Standard 3); die Prozesse werden zwischen ihnen geteilt (8 → 4 + 4 → 3 + 3 + 2).

Die Karte rechts: ALL RENDERS DONE IN

Wann alle laufenden oder wartenden Renders fertig sein sollten, beim Tempo, mit dem der PC gerade rendert. Darunter: wann sie enden, wie lange sie schon laufen und wie viele Frames übrig sind. Sind alle durch, zeigt die Karte **ALL RENDERS TOOK**: wie lange alles gedauert hat.

Die Steuerzeile

Element	Was es tut
Render	Startet den gewählten Render (oder mehrere gewählte). Frames, die schon in Ordnung auf der Platte sind, überspringt er. Nach Stop geht der Render dort weiter, wo er aufgehört hat (Resume). Hat der Render Frames auf der Platte (fertig, gestoppt oder mit Frames von früher), fragt er: **Only missing** rendert nur fehlende oder kaputte Frames (die anderen sind in der Karte dunkelgrün), **All frames again** rendert alles neu, **Cancel** nichts.
Pause	Friert alle Nuke-Prozesse des Renders mitten im Frame ein (die CPU wird frei, der Speicher bleibt belegt). Ein weiterer Klick macht weiter.
Stop	Stoppt den Render, beendet die Prozesse und löscht halb geschriebene Frames und Temp-Dateien von Nuke. Fertige Frames bleiben.
Slate	Ob ein Slate-Frame gerendert wird und auf welchem Frame (Settings › Job).
Framerange	Die Frames dieses Renders: der Bereich aus Nuke, sofern du ihn nicht in Settings › Job geändert hast.
Processes · Threads · Cache · Chunk	Womit der Render läuft. Geändert in Settings › Processes and Chunks; die Zahl der Prozesse auch während er läuft, mit Rechtsklick in System.
Farbiger Balken	Das Ergebnis des Renders, siehe unten.

Der farbige Balken mit dem Ergebnis

Farbe	Bedeutung	Beispiel
Grün	Fertig, alle Frames sind auf der Platte und haben die Prüfung bestanden.	Done in 2:06. All 97 frames are on disk and pass the check.
Gelb	Fertig, aber etwas ist nicht geprüft oder sieht verdächtig aus (Frames nicht geprüft, verdächtig klein; ein Slate gilt nicht als klein).	frames not checked · 2 suspiciously small
Rot	Ein Fehler: der Render ist fehlgeschlagen, kaputte Frames, oder Check pixels hat ein Problem gefunden (NaN, schwarzes Frame…).	2 broken frames · Output: Problems shows them
Grau	Läuft gerade: *Checking pixels…*, Warten auf freie Prozesse und Ähnliches.	Waiting: 1 render already uses the PC

5 Kacheln und Frame-Karte



Kachel	Was sie zeigt
Frames	Fertige / alle Frames des Renders, Prozent und wie viele gerade gerendert werden.
Elapsed	Wie lange der Render läuft (ohne Pausen).
Remaining	Geschätzte Restzeit beim aktuellen Tempo.
Speed	Frames pro Sekunde und pro Minute, alle Prozesse zusammen. Die beste Zahl, um Einstellungen zu vergleichen.
Frame time	Wie lange ein Frame in einem Prozess dauert und wie viele Frames gleichzeitig laufen (× N at once).
Nuke processes	Wie viele Prozesse rendern / laufen, dazu die, die laden oder warten.
Check frames	Die Dateiprüfung: fehlende, leere, abgeschnittene, unlesbare und verdächtig kleine Frames.
Check pixels	Die Pixelprüfung (EXR, PNG, JPEG, TIFF, DPX): schwarze, leere und einfarbige Frames, Frames gleich einem anderen und ein Frame, dessen Größe, Kanäle oder Bittiefe von den übrigen abweichen. Bei EXR auch NaN, inf und Kompression.

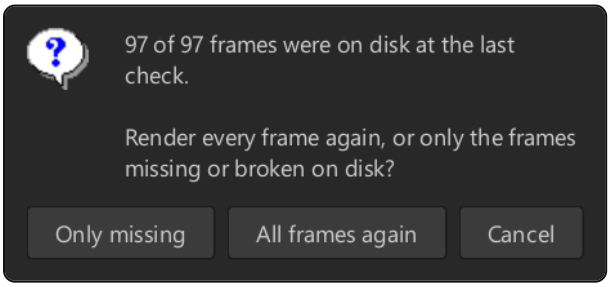
Die Frame-Karte



Jedes Kästchen ist ein Frame, gruppiert nach Chunks (C01, C02…). Ein **Klick auf ein Frame** öffnet es in der Vorschau. Farben:

wartet (queued) wird gerendert fertig war schon auf der Platte, übersprungen (Only missing) verdächtig klein fehlgeschlagen / kaputt

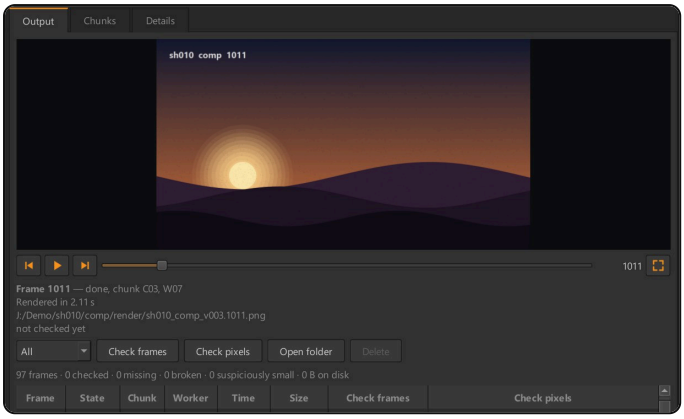
Render again: alles oder nur das Fehlende



Hat der Render Frames auf der Platte und du klickst **Render again**, fragt er: **Only missing** (Standard) rendert nur fehlende oder kaputte Frames; die anderen werden übersprungen und sind in der Karte **dunkelgrün**. **All frames again** rendert alles neu, **Cancel** nichts. Ein Film (MOV, MP4) fragt nicht: ein Film ist eine Datei und wird ganz gerendert.

Mit mehreren per **Shift** gewählten Karten fragt er einmal für alle (z. B. *27 of 27 frames of 3 renders*…). Frames, die beim Öffnen schon auf der Platte waren oder aus dem Lauf vor dem Stop stammen, sind dunkelgrün. In Output zeigt der Filter **Rendered** dann nur die Frames dieses Laufs, **All** alle.

6 Panel Output: Vorschau und Frames



Vorschau

- **Mausrad**: Zoom, **Ziehen** verschiebt das Bild.
- **Doppelklick**: zurück zum ganzen Frame. Ist es schon ganz, wird die Vorschau über alle drei Panels groß; zurück mit `Esc`.
- Beim Zoomen lädt die volle Auflösung (auch EXR und 4K).
- `Space` startet und stoppt das Abspielen. Es drückt nie eine Taste, stoppt also nie den Render.

Die Leiste unter der Vorschau

Voriges Frame · **Play/Pause** (spielt die fertigen Frames in Schleife mit den fps des Scripts) · nächstes Frame · Schieberegler über alle Frames · Frame-Nummer · **vergrößern** (Vorschau über drei Panels, `Esc` zurück). Unter der Leiste: Pfad, Größe, Auflösung, Kanäle und Prüfergebnisse des Frames.

Tasten

Element	Was es tut
Filter	**All**: alle Frames auf einmal: gerendert, übersprungen, kaputt, *not on disk* und noch zu rendernde, dazu Frames auf der Platte außerhalb der Framerange. Die anderen Filter sind Teile davon: **Rendered**: nur in diesem Lauf gerenderte Frames. **Problems**: Fehler, verdächtig kleine Frames und Funde der Pixelprüfung. **Framerange**: nur die Frames des gewählten Bereichs.
Check frames	Prüft die Dateien aller Frames erneut: vorhanden, nicht leer, ganz und lesbar, nicht viel kleiner als die übrigen. Ein Frame, das noch nicht auf der Platte ist, bleibt *not on disk* und gilt nicht als kaputt.
Check pixels	Prüft im Hintergrund die Pixel jedes Frames (EXR, PNG, JPEG, TIFF, DPX): schwarze, leere und einfarbige Frames, Frames gleich einem anderen irgendwo in der Sequenz, abweichende Größe, Kanäle oder Bittiefe; bei EXR auch NaN, inf und Kompression. Etwa 0,1 s pro 4K-Frame. Meldet nur, rendert nichts neu.
Open folder	Öffnet den Ausgabeordner.
Delete	Löscht gerenderte Frames von der Platte: alle (Standard) oder einen angegebenen Bereich. Fragt vorher.

Die Frame-Liste

Spalten: **Frame**, **State**, **Chunk**, **Worker** (welcher Prozess es gerechnet hat), **Time**, **Size**, **Check frames**, **Check pixels**. Mit dem Filter All stehen alle Frames von Anfang an in der Liste, ihr Status ändert sich, während sie gerendert werden. Nach **Render again** verschwinden Frames nicht: sie bleiben als *on disk, queued again* mit alter Größe und Zeit (zum Ansehen und Abspielen), bis der neue Render sie überschreibt. Frames auf der Platte außerhalb der gewählten Framerange bleiben mit dem Vermerk *not in Framerange* in der Liste.

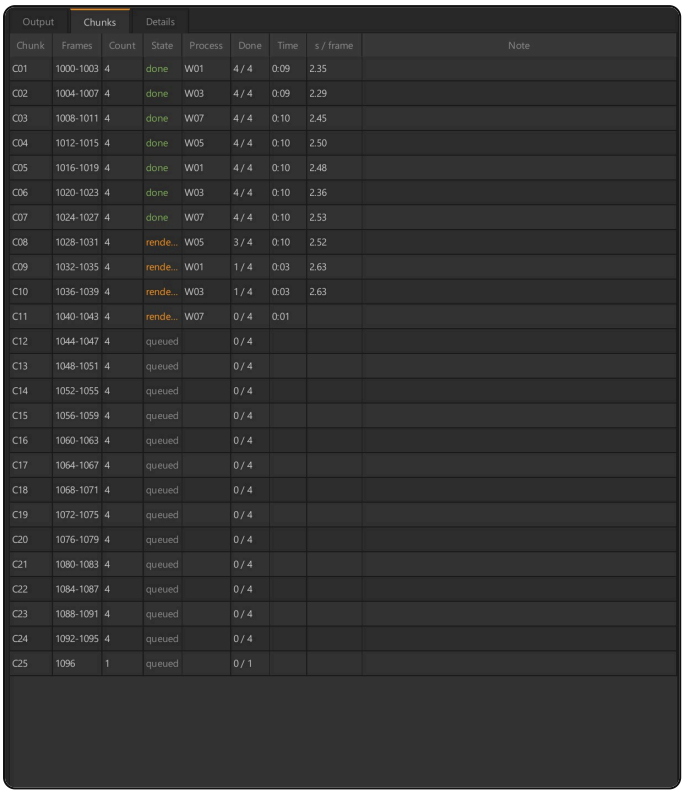


Rechtsklick auf ein Frame (oder mehrere gewählte):
Render this frame / batch: rendert nur diese Frames neu (läuft der Render, kommen sie hinzu).
Delete this frame / batch: löscht sie von der Platte (nur wenn der Render nicht läuft).

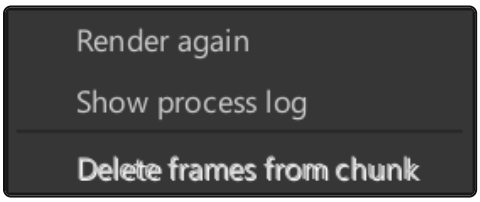
> **MOV / MP4 / MXF:** statt der Frame-Liste erscheint eine Filmdatei mit Details (Codec, Profil, fps, Auflösung, Timecode, Frame-Anzahl gegen die Einstellungen des Writes) und ein Player mit Lupe. Tasten Check video, Open folder, Delete video. Ein Film lässt sich nicht teilen, ihn rendert ein Prozess mit allen Threads; so zeigen es auch die Settings (Processes 1, gesperrt). Die Einstellungen der anderen Renders ändern sich dadurch nicht.

7 Panels Chunks und Details

Chunks



Eine Zeile pro Chunk: **Chunk** (C01…), **Frames** (Bereich), **Count**, **State**, **Process** (wer ihn rechnet), **Done**, **Time**, **s / frame** und **Note** (warum ein Chunk wiederholt wurde und Ähnliches).



Rechtsklick: Render again (den Chunk neu), **Show process log** (Log des Prozesses, der ihn gerechnet hat), **Delete frames from chunk** (nur wenn der Render nicht läuft).

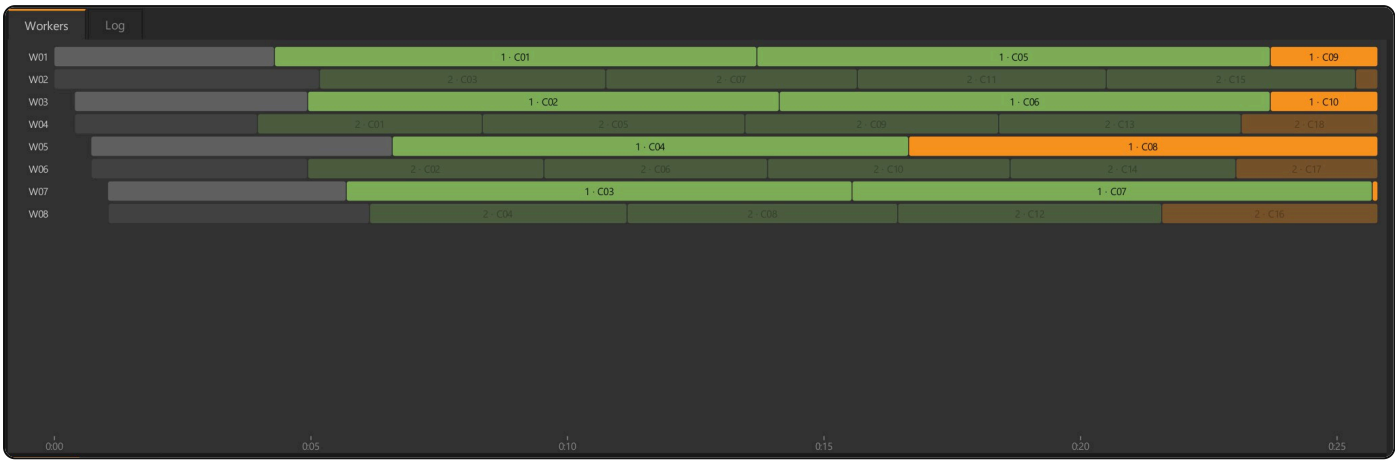
Details



Alles über den Render an einem Ort: das Script und sein Snapshot, die Writes und Ausgabepfade, Format, fps, Bereich, Nuke-Version, mit welchen Processes × Threads × Cache er läuft, Zeiten und die Pfade zu den Logs. Praktisch, wenn du genau wissen willst, was gerendert wurde.

8 Panels Workers und Log

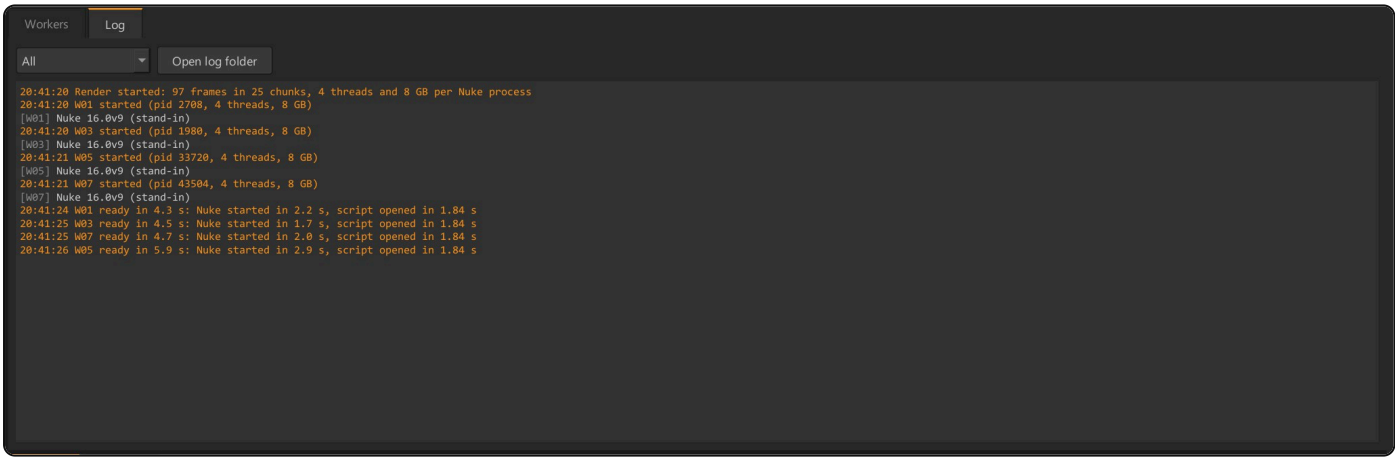
Workers: Zeitleiste der Prozesse



Eine Zeile pro Nuke-Prozess (W01–W08), die Zeit läuft von links nach rechts. ⬛ **grau**: Nuke startet und öffnet das Script (auch beim Tausch gegen ein frisches Nuke, beschriftet *again: fresh Nuke*). 🟩 **grün**: fertige Chunks (mit Chunk-Nummer). 🟧 **orange**: der Chunk, der gerade gerendert wird. Chunks anderer Renders sind gedimmt (Settings › *Dim other renders in Workers*). Man sieht sofort, ob ein Prozess steht, neu startet oder lange das Script öffnet.

Ein Rechtsklick auf die Zeile eines Prozesses öffnet dasselbe Menü wie im Panel System (Kapitel 9).

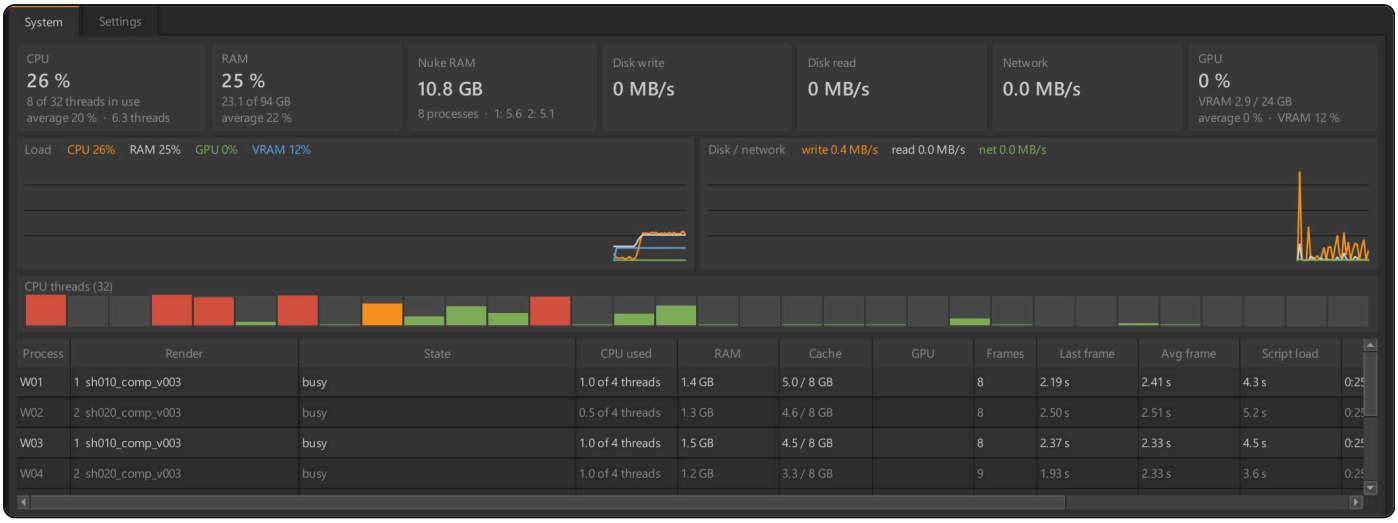
Log



Meldungen von JKrender und die Ausgabe jedes Nuke-Prozesses mit Uhrzeit. Filter: **All**, **JKrender only** (nur Meldungen von JKrender), **Errors and warnings**. Open log folder öffnet den Ordner mit den Logs dieses Renders: Snapshot des Scripts, `job.json` und das `worker_NN.log` jedes Prozesses.

> Jeder Prozess meldet im Log, wie lange er bis zur Bereitschaft brauchte, aufgeteilt in **Start von Nuke** und **Öffnen des Scripts**. Dauert der Start 15 s oder mehr, warnt JKrender. Ein langsamer Start liegt meist an der Lizenz oder den Start-Scripts der Pipeline, nicht am Script selbst.

9 Panel System



Kacheln und Graphen

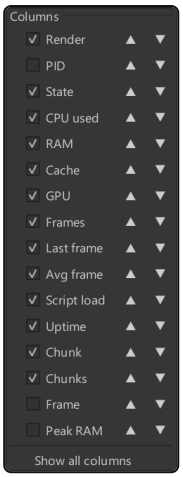
CPU (Auslastung und Threads), **RAM** (gesamt), **Nuke RAM** (was die Nuke-Prozesse belegen), **Disk write**, **Disk read**, **Network**, **GPU** (Auslastung und VRAM). Der Graph **Load** zeigt CPU, RAM, GPU und VRAM in %, der Graph **Disk / network** Übertragungen in MB/s. **CPU threads**: ein Balken pro CPU-Thread, so sieht man, ob der ganze PC ausgelastet ist.

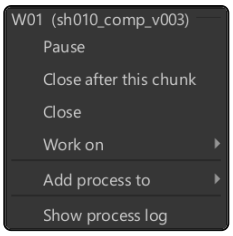
Die Tabelle der Prozesse

Spalte	Bedeutung
Process	Nummer des Prozesses (W01…).
Render	An welchem Render er arbeitet.
State	Was er tut: loading, busy, idle, paused, closing… *busy → fresh Nuke (RAM 91 %)*: nach seinem Chunk bekommt er ein frisches Nuke, weil das RAM am Limit ist (Kapitel 13). Worauf er wartet, z. B. *wait: VRAM 75 → 14 % (3 starting)*: jetzt 75 %, ein weiterer Prozess passt unter 14 %, weil 3 Prozesse noch starten und JKrender ihnen Platz freihält; der Tooltip sagt mehr.
CPU used	Wie viele CPU-Threads der Prozess wirklich nutzt.
RAM · Cache	Der tatsächliche Speicher des Prozesses und sein Nuke-Cache gegen das Limit.
GPU	Auslastung der Grafikkarte und VRAM dieses Prozesses.
Frames · Last frame · Avg frame	Wie viele Frames er gerechnet hat, wie lange das letzte dauerte und im Schnitt.
Script load	Wie lange Nuke zum Starten und Öffnen des Scripts brauchte.
Uptime · Chunk · Chunks	Wie lange er läuft, an welchem Chunk er ist und wie viele er gemacht hat.
PID · Frame · Peak RAM	Versteckte Spalten: Prozess-ID in Windows, aktuelles Frame, höchstes RAM.

Ein Klick auf den Tabellenkopf öffnet die Liste der Spalten: Haken zeigt oder versteckt eine Spalte, die Pfeile ändern die Reihenfolge. Spalten lassen sich auch am Kopf ziehen. Breiten und Reihenfolge werden beim Schließen des Fensters gespeichert.

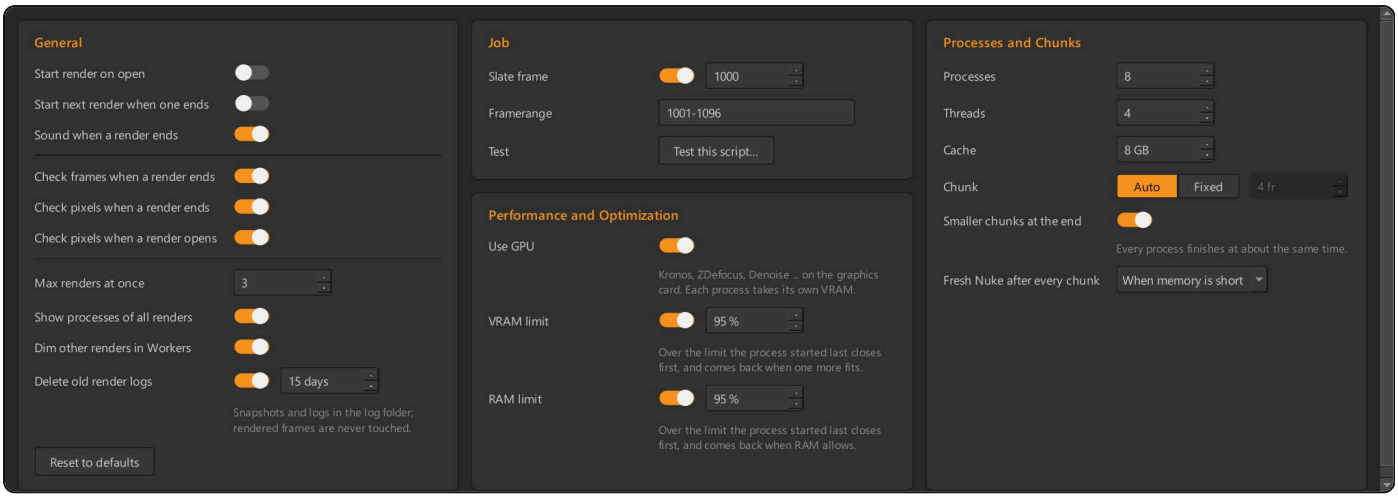
Ein Rechtsklick auf einen Prozess (in der Tabelle oder auf einer Zeile in Workers) öffnet das Menü rechts. Laufen mehrere Renders, hat *Add process to* ein Untermenü zur Wahl des Renders.





Menüeintrag	Was er tut
Pause / Resume	Hält nur diesen Prozess an oder lässt ihn weiterlaufen.
Close after this chunk	Der Prozess beendet seinen Chunk und schließt sich. **Processes** sinkt um 1 und bleibt so.
Close	Schließt den Prozess sofort. Sein Chunk geht zurück in die Warteschlange. An seine Stelle kann ein anderer treten.
Work on › Render	Verschiebt den Prozess auf einen anderen laufenden Render.
Add process	Startet einen weiteren Prozess (auch per Klick auf eine leere Stelle der Tabelle). **Processes** steigt um 1 und bleibt so. Die Limits für RAM und VRAM gelten weiter.
Start again	Bei einem geschlossenen Prozess: startet ihn neu.
Show process log	Öffnet das Log dieses Prozesses.

10 Settings: jede Einstellung



Einstellungen werden sofort gespeichert. Jede hat einen Tooltip. **Reset to defaults** setzt alles auf den Auslieferungszustand zurück, auch die Tabellenspalten; was JKrender über Scripts gelernt hat, bleibt.

General

Einstellung	Was sie tut	Standard
Start render on open	Der Render startet, sobald das Fenster aufgeht, ohne Klick auf Render.	aus
Start next render when one ends	Endet oder scheitert ein Render, startet von selbst der nächste noch nicht gestartete in der Reihenfolge der Karten. Ein Render, den du gestoppt hast, startet nichts.	aus
Sound when a render ends	Ein Ton am Ende des Renders, sobald die eingeschalteten Prüfungen durch sind. Ein anderer Ton bei Fehler, kaputten Frames oder einem Problem in den Pixeln. Ein gestoppter Render bleibt still.	an
Check frames when a render ends	Am Ende des Renders (auch nach Stop) jedes gerenderte Frame einmal: vorhanden, nicht leer, ganz und lesbar, nicht viel kleiner als die übrigen. Während er rendert, liest nichts den Ausgabeordner. Aus: ein kaputtes Frame gilt als fertig, bis du Check frames drückst.	an
Check pixels when a render ends	Gleich nach der Frame-Prüfung im Hintergrund die Pixel aller Frames (schwarz, einfarbig, gleich, anderes Format; bei EXR auch NaN und inf). Meldet nur.	an
Check pixels when a render opens	Dieselbe Prüfung der Frames, die schon auf der Platte sind, sobald ein Render aufgeht. Render stoppt sie.	an
Max renders at once	**1**: Renders laufen nacheinander; der nächste lädt schon, während der vorige seine letzten Chunks macht (der erste Shot ist zuerst fertig). **2–3**: sie laufen nebeneinander und teilen sich die Prozesse (insgesamt nicht schneller, aber von jedem Shot siehst du früh Frames).	3
Show processes of all renders	Workers und System zeigen die Prozesse aller Renders. Aus: nur des gewählten.	an
Dim other renders in Workers	Auf der Zeitleiste leuchten die Chunks des gewählten Renders, die anderen sind gedimmt.	an
Delete old render logs	Beim Öffnen von JKrender löscht es Log-Ordner, die älter als die eingestellten Tage sind. Nie Frames.	an, 15 Tage

Dünne Linien trennen die Gruppen in General. Beim erneuten Rendern zeigt die Zeitleiste in Workers immer nur den neuen Lauf.

Job (gilt für den oben gewählten Render)

Einstellung	Was sie tut	Standard
Slate frame	Das Projekt beginnt nach einem Slate (z. B. 1001, Slate auf 1000). An: jeder Render rendert zusätzlich den Slate auf diesem Frame.	an, 1000
Framerange	Die Frames des Renders, z. B. `1001-1100` oder `1001-1010,1050`. Jeder Render hat seinen eigenen. Ein leeres Feld holt den Bereich aus Nuke zurück. Bestätigen mit Enter oder Klick woandershin. Während des Renders gesperrt. Frames auf der Platte bleiben.	aus Nuke
Test › Test this script…	Misst Einstellungen für diesen PC auf Frames aus dem ganzen Shot und zeigt die schnellste (Kapitel 12).	—

Performance and Optimization

Einstellung	Was sie tut	Standard
Use GPU	Startet Nuke mit `--gpu`, so rechnet die Grafikkarte GPU-Nodes (Kronos, ZDefocus, Denoise…). Ein leichtes 4K-Script mit Kronos war 3× schneller. Jeder Prozess hat sein eigenes VRAM.	an
VRAM limit	Über dem Limit laufen weniger Prozesse. Bei 97 % schließt sofort der zuletzt gestartete Prozess; ebenso, wenn die Karte voll ist und lange kein Frame fertig wird. Innerhalb von 5 % unter dem Limit hilft Fresh Nuke after every chunk. Kapitel 13.	an, 95 %
RAM limit	Über dem Limit laufen weniger Prozesse: zuerst schließt der zuletzt gestartete. Wird Speicher frei, kommt er zurück. So fängt Windows nie an zu swappen.	an, 95 %

Processes and Chunks

Einstellung	Was sie tut	Standard
Processes	Nuke-Prozesse für alle Renders zusammen.	8
Threads	Threads pro Prozess (`-m`).	4
Cache	Cache-Limit von Nuke pro Prozess in GB (`-c`). Was ein Prozess wirklich nimmt, zeigt System › RAM. Ein zu kleiner Cache bremst den Render: Nuke wirft dann weg und rechnet neu.	nach RAM
Chunk: Auto / Fixed	**Auto**: etwa 4 Chunks pro Prozess, höchstens 10 Frames (46 Frames mit 8 Prozessen → 2, 96 → 3, 320+ → 10). **Fixed**: die Größe stellst du rechts ein. Gilt für Chunks, die noch nicht begonnen haben.	Auto
Smaller chunks at the end	Zum Ende des Renders werden die Chunks kleiner bis zu einzelnen Frames, so werden alle Prozesse etwa gleichzeitig fertig.	an
Fresh Nuke after every chunk	Nach seinem Chunk schließt ein Prozess Nuke und öffnet das Script in einem neuen, mit RAM und VRAM wieder am Anfang. Das kostet ein Laden des Scripts. **When memory is short**: nur wenn RAM oder VRAM 30 s am Stück innerhalb von 5 % unter dem Limit sind, und zwar ein Prozess nach dem anderen. **Always**: nach jedem Chunk. **Off**: nie. Kapitel 13.	When memory is short

> Processes, Threads und Cache sind während eines Renders gesperrt. Bei einem Film zeigen sie 1 Prozess mit allen Threads. Die Zahl der Prozesse änderst du während des Renders mit Rechtsklick in System (*Add process*, *Close after this chunk*).

11 Einen Render einstellen

Empfohlener Weg bei einem neuen oder schweren Script:

1	2	3	4	5
Write senden Ctrl+Alt+R	**Create tests** Einstellungen nach PC	**Test starten** Round 1 und Round 2	**Use these settings** übernimmt die grüne Zeile	**Render** und System beobachten

Schritte 2–4 sind im Fenster Test this script. Leichte Scripts ruhig gleich mit 8 × 4; ein Test lohnt sich, wenn ein Render Dutzende Minuten dauert.

Richtwerte (Ryzen 9950X, 32 Threads, 94 GB RAM, RTX 3090 24 GB)

Art des Scripts	Processes × Threads	Warum	Gemessen
Leichter Comp, Keying, Grade	**8 × 4**	Ein Nuke nutzt nicht alle Threads, mehr Prozesse schon.	4K-Testscript: 8 × 4 am schnellsten
Schwerer Comp (Defocus, Denoise, viele Ebenen)	**4 × 8** oder **2 × 16**	Ein Prozess nutzt mehr Threads. 8 Prozesse erschöpfen das RAM, Prozesse schließen.	schwerer Comp: 4 × 8 mit 12 GB Cache 17 fr/min, 2 × 16 16 fr/min
Script, dessen VRAM wächst (GPU-Nodes)	**1–2 × 32/16**	Das VRAM wächst mit jedem Frame; mehr Prozesse passen nicht auf die Karte.	VRAM +1,7 GB pro Frame → 1 × 32

Woran man eine schlechte Einstellung erkennt

Zu wenig Speicher

System › RAM ist am Limit, in State steht *wait: RAM …*, Prozesse schließen und starten neu (graue Blöcke in Workers). **Lösung:** weniger Prozesse mit mehr Threads oder ein kleinerer Cache.

Volle Grafikkarte

VRAM bei 95 %, Prozesse schließen wegen VRAM. JKrender schlägt selbst weniger Prozesse mit mehr Threads vor (ändert aber nichts von sich aus). **Lösung:** weniger Prozesse mit mehr Threads (Test this script zeigt wie viele), oder Use GPU ausschalten.

Ungenutzte CPU

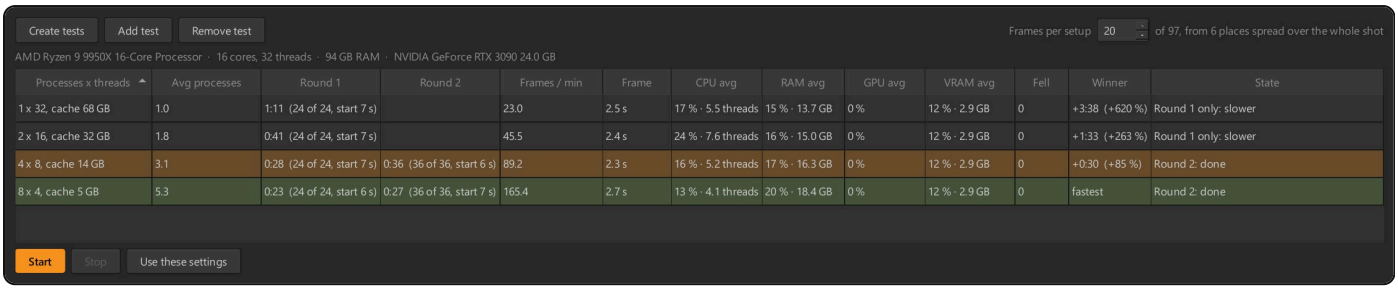
CPU threads sind großteils leer und das RAM hat Reserve. **Lösung:** einen Prozess hinzufügen (Rechtsklick › Add process) oder Threads erhöhen.

Langes Öffnen des Scripts

Graue Blöcke in Workers sind lang, Script load ist hoch. Beim Start streiten die Nukes um Lizenzen und Start-Scripts. **Lösung:** längere Chunks fallen weniger ins Gewicht; mit dem Script selbst hat es meist nichts zu tun.

> **Das beste Maß ist die Kachel Speed** (Frames pro Minute). Vergleiche sie zwischen Einstellungen am selben Script oder überlasse das *Test this script*.

12 Test this script



Ein fertiger Test: die grüne Zeile ist die schnellste, die orange die zweite. Winner zeigt, wie viel länger der ganze Shot mit den anderen dauern würde.

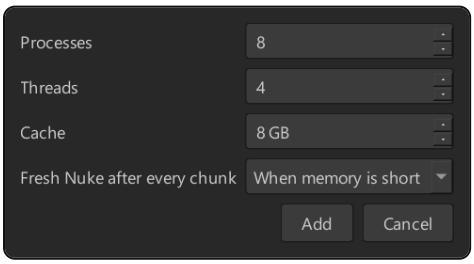
Tasten oben

Create tests schaut sich den Computer an (Zeile unter den Tasten: CPU, Kerne und Threads, bei Intel Performance- und Effizienzkerne, RAM, Grafik) und erstellt **7 Einstellungen** von 1 Prozess mit allen Threads bis zu so vielen Prozessen, wie der PC verträgt: jeder Prozess mindestens 4 Threads und mindestens 4 GB Cache. Auf 32 Threads und 94 GB: 1×32, 2×16, 3×10, 4×8, 5×6, 6×5, 8×4. Über 64 Threads bekommt ein Prozess höchstens 64 (eine Prozessorgruppe in Windows). Ersetzt alles in der Tabelle.

Add test fügt eine eigene Einstellung hinzu (Processes, Threads, Cache und Fresh Nuke after every chunk), Remove test entfernt die gewählten. **Rechtsklick** auf eine Zeile: **Edit test**. Eigene Tests werden gemerkt. Die Tabelle zeigt Fresh Nuke im Namen der Einstellung: *4 x 8, cache 12 GB, fresh every chunk* oder *never fresh*; ohne Zusatz ist es When memory is short.

Der **Cache** jeder Einstellung ist das ganze RAM für Nuke (RAM-Limit, abzüglich einer Reserve für Windows) geteilt durch die Zahl der Prozesse, minus 4 GB, die ein Prozess außerhalb des Caches nimmt. Was gerade läuft, spielt keine Rolle.

Frames per setup (oben rechts): wie viele Frames jede Einstellung in Round 2 rendert. Hat der Shot weniger Frames, zeigt es das und rendert alle.



Wie der Test abläuft

Die Frames kommen aus **6 Stellen, verteilt über den ganzen Shot** (vom ersten bis zum letzten Frame), an jeder ein paar Frames hintereinander. **Alle Einstellungen einer Runde rendern dieselben Frames.**

Round 1: jede Einstellung kurz, 3 Frames pro Prozess der Einstellung mit den meisten Prozessen (auf 32 Threads 24 Frames), höchstens 3 Minuten. **Round 2:** die zwei schnellsten (und jede innerhalb von 5 % der besten) erneut mit Frames per setup, mindestens 4 Frames pro Prozess.

Frames / min wird gezählt, sobald die Prozesse laufen: der Start von Nuke und das erste Frame jedes Prozesses zählen nicht; in einem langen Render kommen sie nur einmal vor.

Spalten: **Avg processes** (wie viele Prozesse im Schnitt gerendert haben), **Round 1**, **Round 2** (Zeit der Runde, Frames, Start), **Frame**, **CPU / RAM / GPU / VRAM avg**, **Fell** (abgestürzte Prozesse), **Winner**, **State**. Klick auf den Kopf sortiert, Spalten lassen sich verbreitern und verschieben.

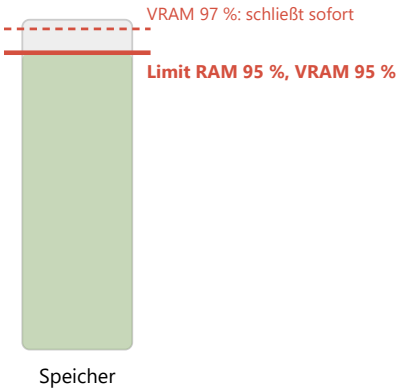
Use these settings übernimmt die grüne Zeile in die Settings, auch Fresh Nuke. Die Frames gehen in einen Testordner und werden gelöscht; Render-Hooks des Writes laufen im Test nicht. Tabellen bleiben in `~/.nuke/JKrender/settings/test_results`.

Das Fenster **Add test** (und dasselbe **Edit test** per Rechtsklick).

> Während eines Tests sollte nichts anderes rendern: der Test würde zwei Renders messen, die sich um den PC streiten. Neben einem Render startet der Test nicht.

13 RAM und VRAM im Blick

JKrender überwacht den Speicher, damit Windows nicht anfängt zu swappen und die Grafikkarte nicht vollläuft.



30 s am Stück innerhalb von 5 % unter dem Limit
Nach ihren Chunks bekommen die Prozesse nacheinander ein frisches Nuke (Fresh Nuke).

Über dem Limit
Der zuletzt gestartete Prozess schließt: sofort, wenn er noch das Script öffnet, sonst nach seinem Chunk.

Rückkehr
Es kommen auf einmal so viele zurück, wie passen; sie starten nacheinander.

Wieder überlastet
Gewartet wird 5 s, dann 30, 60 und 150 s, wenn es sich binnen 10 Minuten wiederholt.

Damit ein Prozess zurückkommt, muss ein Prozess der Größe, die die Prozesse dieses Renders hatten (bis dahin: sein Cache + 4 GB), unter das Limit passen, mit 2 % Reserve und dem, was die laufenden Prozesse noch nehmen werden. Worauf gewartet wird, zeigt die Spalte **State** in System, z. B. *wait: VRAM 75 → 69 %*: das VRAM ist bei 75 % und muss unter 69 % fallen, damit ein weiterer Prozess passt. *wait: VRAM 75 → 14 % (3 starting)*: drei Prozesse starten noch, und JKrender hält ihnen Platz in voller Größe frei. Weitere Zustände: *wait: 25 s* (ein Prozess ist gerade abgestürzt), *wait: W03 1st frame* (wartet auf das erste Frame von W03, um zu sehen, wie viel ein Prozess nimmt), *wait: VRAM ok, checking* (Platz ist da, es prüft, ob er bleibt).

Volle Karte und nichts geht: ist das VRAM über dem Limit und kein Prozess beendet ein Frame länger als 4× die übliche Frame-Zeit (mindestens eine Minute), ist Nuke wohl auf die CPU ausgewichen und alle Prozesse warten. Dann schließt sofort der zuletzt gestartete Prozess und sein Chunk geht zurück in die Warteschlange.

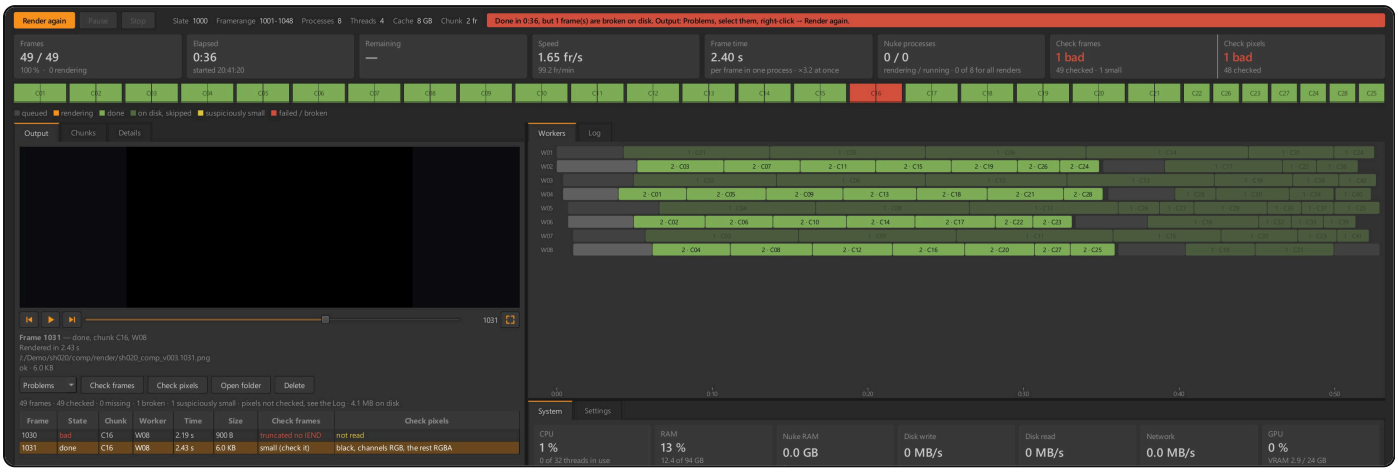
Fresh Nuke after every chunk

Beim Rendern behält Nuke Speicher, den es nicht mehr braucht: ein Prozess eines schweren Scripts kann in seinen ersten ~20 Frames von 2 auf 7 GB VRAM wachsen. Ein frisches Nuke beginnt wieder klein, so passen mehr Prozesse auf die Karte. Im Standard **When memory is short** passiert das nur, wenn RAM oder VRAM **30 s am Stück** innerhalb von 5 % unter dem Limit sind (ein einzelner Ausreißer reicht nicht), und **ein Prozess nach dem anderen**: der nächste erst, wenn das neue Nuke sein erstes Frame gerendert hat und der Speicher immer noch am Limit ist. Die anderen rendern derweil weiter. Ein Prozess, der ein frisches Nuke bekommt, zeigt in State *busy → fresh Nuke (VRAM 93 %)*; bei den letzten Chunks wird nichts getauscht.

Eine VRAM-Grenze pro Prozess gibt es nicht. Cache (`-c`) begrenzt nur Nukes RAM-Cache. Den Grafikspeicher eines Prozesses kann man nicht begrenzen: Windows kann es nicht, und die Blink-Cache-Einstellungen von Nuke wirken im Render nicht (gemessen). Deshalb steuert JKrender das VRAM über die Zahl der Prozesse und frische Nukes. Cache ist ein Cache-Limit, nicht der ganze Speicher eines Prozesses; die echte Zahl steht in System › RAM.

14 Prüfungen, Töne, Absturz und Daten

Prüfung der Ausgabe



Ein Render mit einem abgeschnittenen Frame (1030) und einem schwarzen Frame (1031), beide von Hand mit Check frames und Check pixels gefunden: roter Balken, rote Frames in der Karte, der Filter Problems zeigt nur sie.

Prüfung	Wann	Was sie sucht
Check frames	Am Ende des Renders und nach Stop (abschaltbar) und per Taste. Immer beim Öffnen eines Renders und vor Render.	Fehlende, leere, abgeschnittene und unlesbare Dateien (EXR, PNG, JPEG, DPX), verdächtig kleine Frames.
Check pixels	Am Ende des Renders, beim Öffnen (abschaltbar) und per Taste.	EXR, PNG, JPEG, TIFF und DPX: schwarze, leere und einfarbige Frames, Frames gleich einem anderen, abweichende Größe, Kanäle und Bittiefe. Bei EXR auch NaN, inf und Kompression. Andere Formate: nur gleiche Dateien.
Movie check	Bei MOV, MP4 und MXF.	Codec, Profil, fps, Auflösung, Timecode und Frame-Anzahl gegen die Einstellungen des Writes.

Ton nach dem Render

Nach dem Render laufen die Prüfungen in der Reihenfolge **Check frames** → **Check pixels**; der Ton kommt, sobald die eingeschalteten durch sind (sind beide aus, gleich nach dem Render).

Fröhlicher Ton: alles in Ordnung. **Fehlerton**: der Render ist fehlgeschlagen, hat kaputte Frames oder die Pixelprüfung hat ein Problem gefunden. Ein von Hand gestoppter Render bleibt still.

Wenn JKrender abstürzt

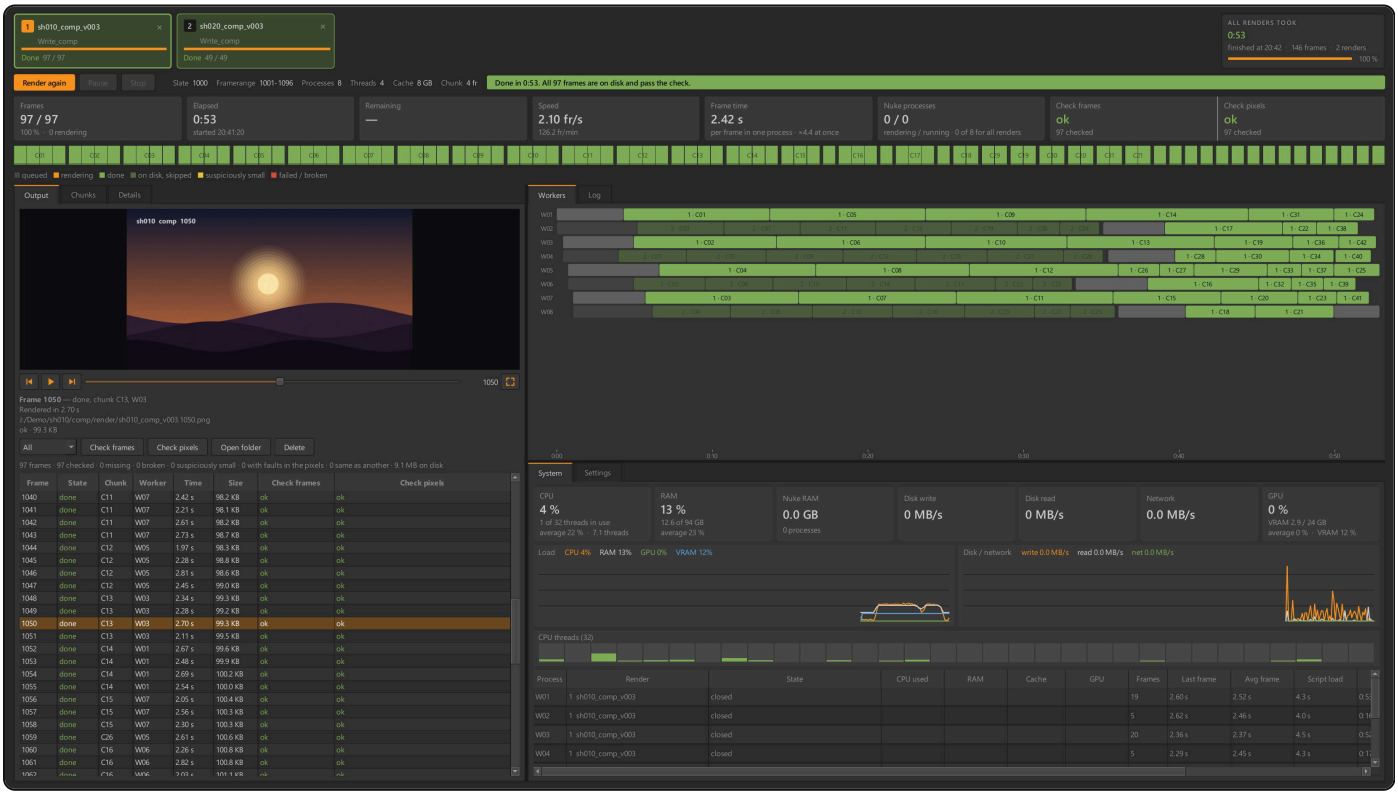
Die Karten kommen beim nächsten Öffnen von JKrender zurück (etwa mit dem nächsten aus Nuke gesendeten Render). Was auf der Platte ist, zählt, übrige Nuke-Prozesse werden beendet. Ein Render, der lief, wartet auf Render und rendert dann, was fehlt. Schließt du das Fenster normal, kommt nichts zurück.

Render-Daten

Jeder Render schreibt in seinen Log-Ordner `report.json`: CPU, RAM und Grafikkarte dieses PCs, Einstellungen, Tempo, Speicherspitzen, Schließen wegen Limits, Start von Nuke, Öffnen des Scripts und die Arten von Nodes im Script. **Darin stehen keine Script-Namen, Pfade oder Fehler**; ein Script ist nur an einer ID erkennbar, die über seine Versionen gleich bleibt. Triffst du auf einen Fehler, schicke den Log-Ordner des Renders.

15 Maus und Tasten: Spickzettel

Wo und was	Ergebnis
`Ctrl+Alt+R` in Nuke	Schickt die gewählten Writes an JKrender.
Klick auf eine Render-Karte	Wählt den Render; das ganze Fenster zeigt seine Daten.
`Shift` + Klick auf Karten	Wählt mehrere Renders für Render, Render again, Pause, Stop.
Klick auf ein Frame in der Frame-Karte	Öffnet das Frame in der Vorschau.
`Space`	Vorschau abspielen / anhalten (stoppt nie den Render).
Rad / Ziehen in der Vorschau	Zoom / Bild verschieben.
Doppelklick in der Vorschau	Ganzes Frame; ist es schon ganz, große Vorschau.
`Esc`	Zurück aus der großen Vorschau.
Rechtsklick auf ein Frame (Output)	Render this frame / batch, Delete this frame / batch.
Rechtsklick auf einen Chunk (Chunks)	Render again, Show process log, Delete frames from chunk.
Rechtsklick auf einen Prozess (System, Workers)	Pause, Close after this chunk, Close, Work on, Add process, Start again, Show process log.
Rechtsklick auf eine leere Stelle in System	Add process.
Klick auf den Kopf der Tabelle System	Auswahl und Reihenfolge der Spalten.
Ziehen einer Panel-Grenze	Ändert die Größe der Panels (wird gespeichert).
Framerange + `Enter`	Bestätigt einen neuen Bereich (Settings › Job).



Ein fertiger Render: grüner Balken, die Karte oben rechts zeigt, wie lange die Renders gedauert haben.

16 Wenn etwas nicht geht

Problem	Was tun
Nuke-Prozesse starten lange (30–50 s)	Meist der Start von Nuke (Lizenz, Start-Scripts der Pipeline), während der PC rendert. Das Log teilt es auf: *Nuke started in …* / *script opened in …*.
Prozesse schließen und starten neu	RAM oder VRAM ist ausgegangen (State in System, Log). Weniger Prozesse mit mehr Threads oder ein kleinerer Cache; Test this script zeigt, was am schnellsten ist.
In State steht *busy* → *fresh Nuke*	RAM oder VRAM ist seit 30 s am Limit; nach seinem Chunk bekommt der Prozess ein frisches Nuke (Settings › Fresh Nuke after every chunk). Passiert das ständig, laufen mehr Prozesse, als passen.
Der ganze Render ist langsamer als letztes Mal	Prüfe den Cache: zu klein (z. B. 4 GB bei einem schweren Script) heißt, Nuke wirft weg und rechnet neu, und Frames dauern ein Mehrfaches.
Das Script öffnet in Nuke mit einem Knob-Fehler	Z. B. *LensDistortion2.serialiseKnob: Failed to load* (ein Node aus einer anderen Version, ein fehlendes Plug-in). JKrender rendert weiter, wie Nuke im GUI; die Meldung steht im Log des Prozesses.
Gelber Balken „not checked"	Die Frame-Prüfung ist aus (Check frames when a render ends) oder lief nicht. Drücke Check frames.
Roter Balken	Output › der Filter Problems zeigt, welche Frames. Rechtsklick *Render this frame / batch*. Der Balken wird auch nach Check frames und Check pixels von Hand neu berechnet.
Keine EXR-Vorschau	numpy/OpenEXR fehlen. Lösche `lib/DECLINED` und starte Nuke neu, oder installiere sie laut README.
Der Render startete nicht sofort	Status Waiting: andere Renders nutzen schon den PC (Max renders at once), oder er wartet auf Speicher.
Leeren von Nukes Cache geht nicht	Nach einem Update von JKrender Nuke einmal neu starten.
Unklar, was gerendert wurde	Das Panel Details und Log › Open log folder (Snapshot des Scripts und job.json).

> **JKrender ist eine Alpha-Version.** In Tests funktioniert es, aber rechne mit Kleinigkeiten. Triffst du auf einen Fehler, schicke dem Autor den Log-Ordner des Renders (Open Renders Folder) und eine Beschreibung, was passiert ist.

JKrender 0.94.3 alpha · github.com/JanKanta/JKrender · Anleitung aktualisiert am 9. 11. 2026. Die Bilder stammen aus einem Beispiel-Render (Shots sh010 und sh020).